RENJI LI

+852 63679767 | handsome-to-the-point-of-death.li@connect.polyu.hk
github.com/Lirenji2025

SELECTED PORTFOLIO | Spatial Storytelling & Interactive Design

Selected materials for Track 1: architectural heritage VR tour interaction and visual design. The projects below demonstrate spatial experience planning, environmental storytelling and game-mechanic communication.

01 | THE FORSAKEN - PAGES 2-8

Solo first-person narrative exploration Aug - Oct 2025

- Focus: circular village route, encounter pacing, multi-perspective narrative loops, environmental progression and photo-caption interaction.
- Tools documented in the project: UE5, Figma, Photoshop, Maya and Twine.
- Presented as a desktop first-person project; the material does not claim a Quest-headset deployment.

02 | INTO THE ALCHEMIST'S HOUSE - PAGES 9-13

Game and level design | Team of two Jul 2024

- Focus: draggable layers, light/shadow-driven platform states and an 11-level teaching progression for a PC/Web puzzle platformer.
- Includes mechanic breakdowns, level diagrams, gameplay screens and implementation examples from the team project.

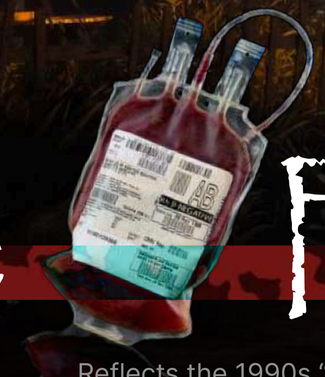
ADDITIONAL WEB-INTERACTION WORK

ORBIT: spatial interaction, persistent local states and feedback-driven iteration using AI-assisted coding.
https://lirenji2025.github.io/Tabs-around-the-earth/
Source code and process documentation

Hong Kong Radiation Map: public-data interaction, time views and visual encoding.
https://github.com/Lirenji2025/Radiation-in-Hong-Kong-urban

The following project pages retain the supplied layouts. AI-assisted implementation is identified where documented; third-party materials remain subject to their original credits.

The Forsaken



Born as weeds, gone as dust.

Reflects the 1990s “**plasma economy**” tragedy; explores human complexity, systemic failure, dignity & kinship; serves as a “**digital memorial**” for the forgotten.

08/2025-10/2025
2 Months (Part-Time, **Solo Developer**)
Figma+PS+Maya+Twine+UE5
First-person/ Walking Simulator / Narrative Exploration

00:00

Game Background

Plasma Economy

In the 1990s, the “plasma economy” in central rural China led to HIV outbreaks in multiple “AIDS villages,” driven by unregulated blood buying and reinfusion practices, causing collective and intergenerational tragedies.

Poverty + misinformation + weak regulation compounded by early information blackout and lack of accountability amplified the disaster.








In one “AIDS village” in Henan, a grandmother who had lost her own children wailed in anguish as she held her grandson, now critically ill after his parents contracted HIV through blood selling. This family tragedy epitomizes how the “blood disaster” entangled individual misfortunes with a collective catastrophe.

In the epidemic triggered by the “plasma economy,” the suffering of a single person was often the microcosm of an entire community’s collapse: whole villages, hundreds of farmers, were swept into the same tragedy in similar ways. Within just a few years, countless villagers fell ill and died, leaving behind orphans and elderly survivors struggling in hardship. In these “AIDS villages,” individual fate and collective disaster were inseparable—each household’s ruin adding to the village’s mournful elegy, an elegy composed of innumerable concrete human tragedies.

Many villagers, once infected, felt both anger at those who led the blood-selling and guilt for passing the virus to their families. In truth, there were no real villains in the “AIDS villages”—harm was mostly unintentional, while the innocent were everywhere. The real cause lay in layers of social and institutional failure, turning a preventable accident into a catastrophe. As one saying goes: “A system where no one is accountable is a system without responsibility.” The tragedy was not only a medical and institutional failure, but also a test of humanity and values. On the edge of life and death, money, dignity, and kinship were re-examined. The calamity exposed prejudice and indifference, but also revealed resilience and kindness. Many patients, even near the end, said: as long as there was respect and companionship, their final days still held dignity and meaning.

https://en.wikipedia.org/wiki/Plasma_Economy

Core Themes

- **Individual fate × collective catastrophe**
  family tragedies aggregate into village history.
- **Human complexity**
  victims may also become (unwilling) transmitters.
- **Social critique**
  systemic flaws, profiteering, and ignorance co-produce harm.
- **Ethical inquiry**
  money, dignity, and kinship re-weighed before death.
- **Commemoration**
  a digital memorial for the forgotten.

Design Intent

This project seeks to recreate the absurdity and tragedy of the real “AIDS villages” through minimalist interaction and fragmented storytelling.

Unlike traditional goal-driven games, The Forsaken treats walking as a form of witnessing, forcing players—through quiet listening and observation—to confront unanswerable questions and moral dilemmas:

- **Who bears responsibility?**
- **Where is the boundary between survival and morality?**
- **How does individual fate get swallowed in collective disaster?**

Narrative Game Paradigms: Space, Loop, Dialogue, and Fictional Identity

To better transform a real-life tragedy into a game experience and evoke player empathy, I have drawn on the design approaches of several outstanding narrative games, leading to the following perspectives for design exploration:

SPACE

Space & Environmental Storytelling

Dear Esther and Rapture show how environments themselves tell stories—ruins, fields, and objects act as both narrative clues and emotional mirrors.



STRUCTURE

Loop & Fragmented Narrative

Rapture uses nonlinear fragments, The Beginner’s Guide employs repetition and interruption. Loops let players relive events with new roles or times, while fragments require them to piece together the full story.



BRIDGE

Dialogue Modes

Dear Esther uses lyrical monologue, Rapture emphasizes overheard conversations, Beginner’s Guide mixes text and meta-narrative. Together they create layers of intimacy, eavesdropping, and interpretation.



STANCE

Fictional Identity & Perspective

Players may embody a survivor, a visitor, or even a ghost. Ambiguity or reframing of identity reinforces witnessing and remembrance, making players carriers of history and memory.

A digital memorial built with minimal interaction and maximum humanity.

Game Overview

The Forsaken is a walking simulator narrative game.

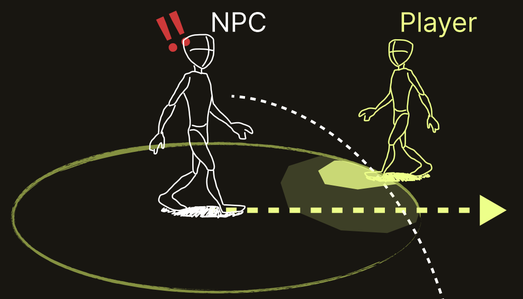
Players wander through a dying village, revisiting the same circular path three times—each loop as a different character. By listening to the last words of ghostly NPCs, observing environmental traces, and capturing photographs with auto-generated captions, players gradually uncover the tragedy of the "AIDS villages" and the fractures within human nature.

This is a story about poverty, disease, responsibility, and forgetting. The game does not offer victory, puzzles, or reward—it offers witnessing and reflection.

Target Experience:
players move through arcs of innocence → anger → repentance → contemplation,
ending with a sense of commemoration and unresolved questions.

Core Verbs

- Walk → advance narrative and time.
- Listen → trigger NPC dialogues.
- Look → observe environment for clues.
- Photo-Caption → capture scenes, generating captions that serve as epitaphs or commentary.

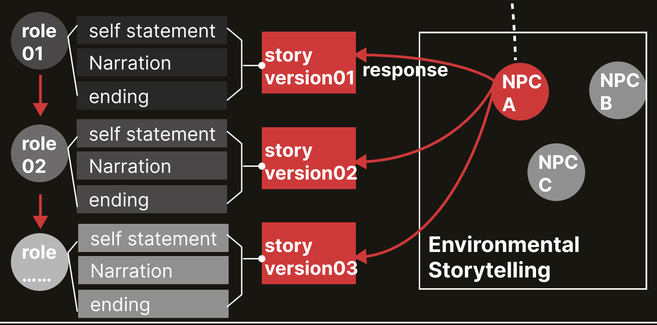


Narrative Structure

Fragmented Nonlinear Storytelling:
players assemble meaning from dispersed NPC dialogues and captions.

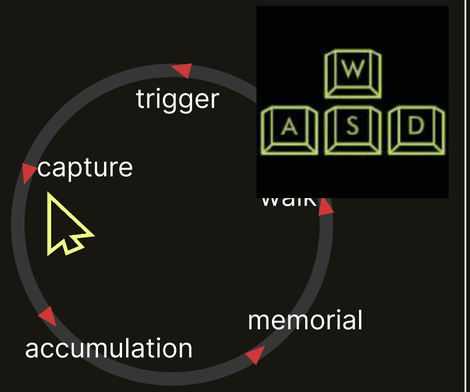
Multiple Identities:
same NPC lines resonate differently depending on the player's current role.

Looped Revelation:
from doubt (child) → exposure (father) → reflection (chief).



Interaction & System Design

NPC Dialogue: auto-trigger on proximity; closing line if player departs early.
Photography: limited shots per loop—forces meaningful choice.
Graveyard: each loop ends by displaying the player's captured photos/captions.



Core Design Framework

Player Change Identity

- Loop 1: The Child (Daytime) → naïve, innocent, poisoned by others.
- Loop 2: The Father (Dusk) → profiteer, "blood boss," the cause of disaster.
- Loop 3: The Village Chief (Night) → witness, bearer of guilt, seeking redemption.

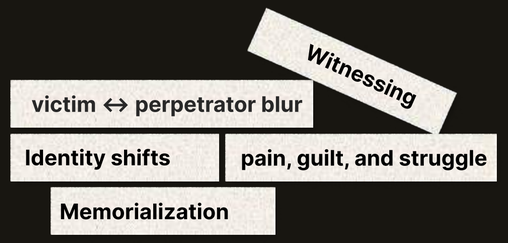
Environmental Shifts

- Child → daylight, hazy innocence.
- Father → sunset, oppressive, accusatory.
- Chief → nighttime, bleak, sorrowful farewell.

NPC Last Words

Each NPC remains frozen at their final position in life. When approached, they speak short dialogues—pleas, accusations, or laments. If the player walks away early, the NPC delivers a closing remark, underscoring unfinished farewells.

Experience Design Pillars



Highlights Design

Role Loop Mechanic	Combines multi-perspective storytelling with gameplay loops, letting players experience "same place, different fate" immersion.
Last Words System	The act of walking away becomes a narrative trigger itself, reinforcing realism and interactivity.
Photo-Narrative Mechanic	Turns photography into commemoration and interpretation, making the player both recorder and narrator.
Layered Environmental Progression	Uses repeated spaces with layered history so the environment itself becomes a timeline.
Based on Real Social Tragedy	Adapts documentary and literary elements into interactive storytelling, where "walking" becomes witnessing.

Level Design

Narrative Support

The map must embody the multi-loop narrative structure. balance repetition (same path each loop) with variation (environmental changes, object accumulation, shifts in mood).

Smooth Experience

The spawn point (graveyard) must be far enough from the first NPC so that the opening monologue plays without interruption.

Each NPC encounter should flow naturally into the next area.

Emotional Pacing

Monologue → NPC → Environmental pause → Next NPC → Graveyard → Next Role → Monologue

Spaces between NPCs allow for reflection and prevent information overload.

Spatial Coherence

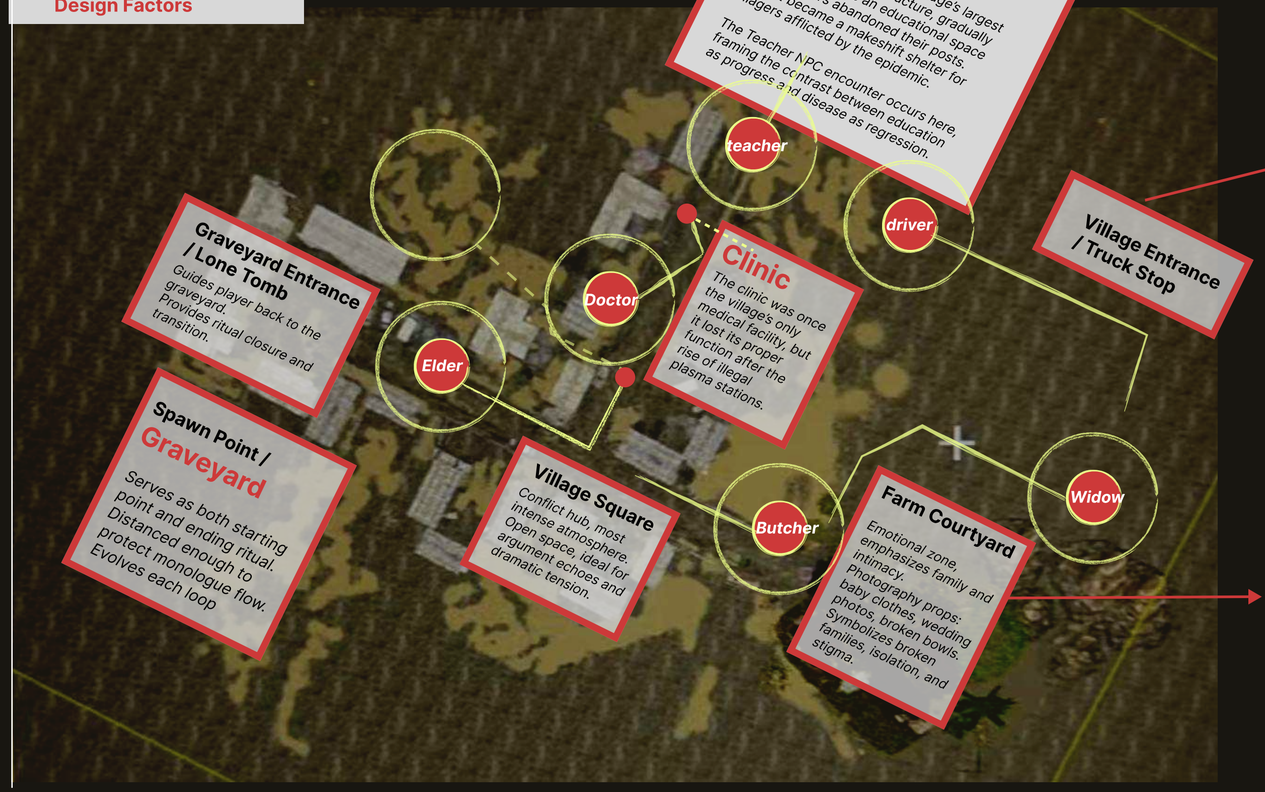
The village must form a clear circular path.

Key locations must be both functional (dialogue triggers) and symbolic (school, clinic, homes, gathering hall, etc.).

References



Design Factors













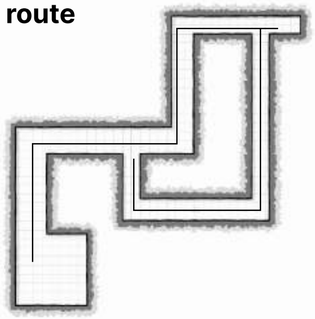




The Forsaken – Mapping Overview

ID	Scene Area	NPC	Scene Props
1	**Village Entrance / Truck**	Driver – (Frank Wang)	Old Liberation truck, faded propaganda banners, empty coffins
2	**Village Entrance / Loudspeaker**	Villager- (Bob Yang)	bamboo pole with loudspeaker, roadside stool
3	**School**	Teacher – (Daniel Ding)	Blackboard with faint chalk, broken desks, straw mats, makeshift sickbeds
4	**Clinic**	Doctor – (Allen Li)	Small clinic hut, dirty white curtains, wooden medicine cabinet, syringes, discarded IV bottles, kerosene lamp
5	**Village Square**	Butcher – (Tom Zhang)	Stone well, wooden benches, broken loudspeaker, abandoned carts, scattered straw mats
6	**Village House**	Villager – (Grace Liu)	Stove, herbal pot, worn quilts, smoke traces
7	**Village House**	Villager – (Victor Zhao)	broken windows, collapsed plaster walls
8	**Graveyard**	death list	Tombstones, wild grass, burned paper offerings, wind chimes

route



Vegetation / Nature

Wild grass / Overgrown weeds
Withered trees
Autumn farmland (dry corn stalks, harvested fields)
Shrubs on barren slopes
Fallen leaves

Village Structures

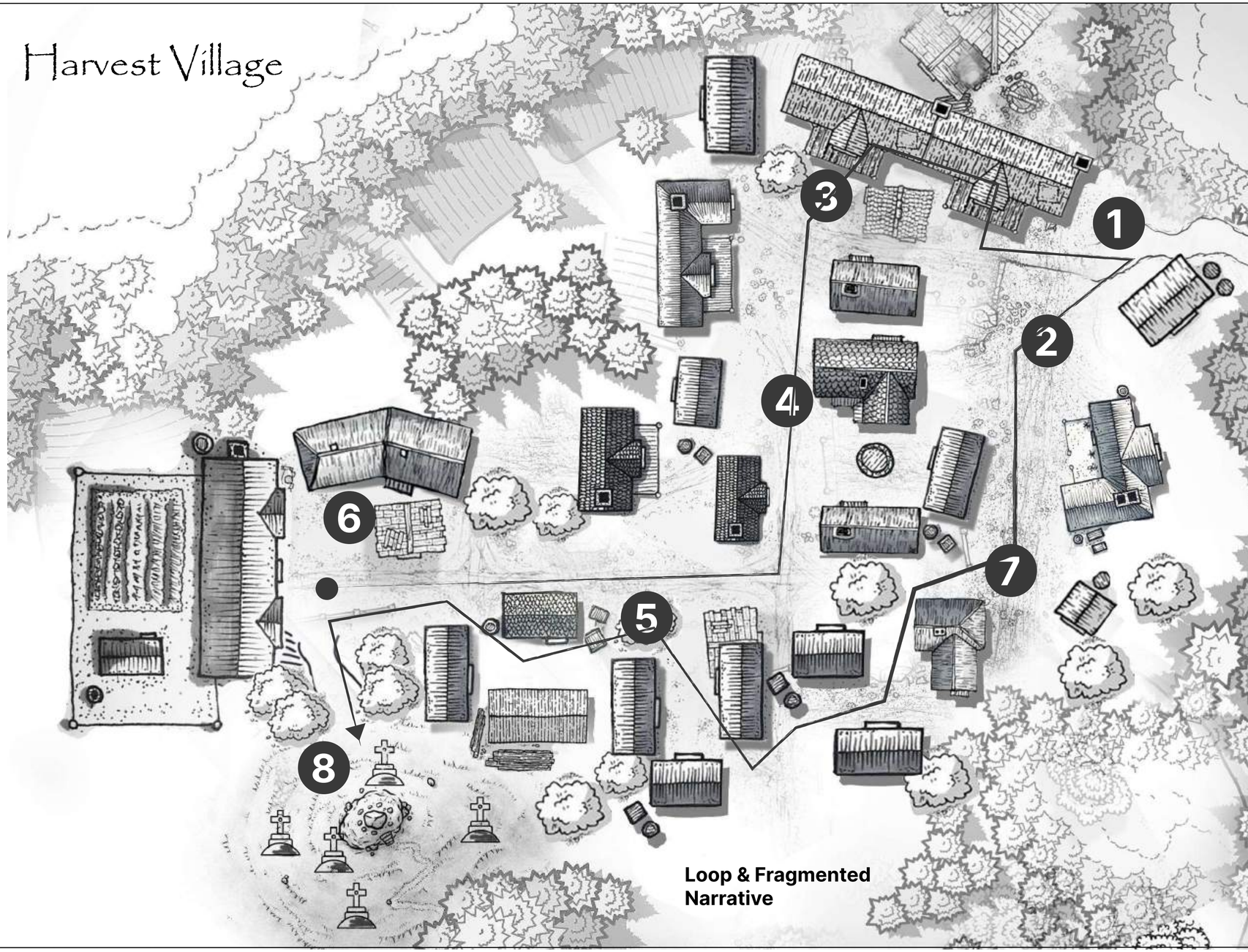
Earthen walls & dirt paths
Stone mill & well mouth
Bamboo fences
Thatched roofs
Wooden doors with rusty locks
Stone stele at village gate

Time & Environmental Shifts

Weather transitions (cloudy → gloomy → clear)
Daytime-realistic tone
Dusk-oppressive
Night-silent, penitential tone

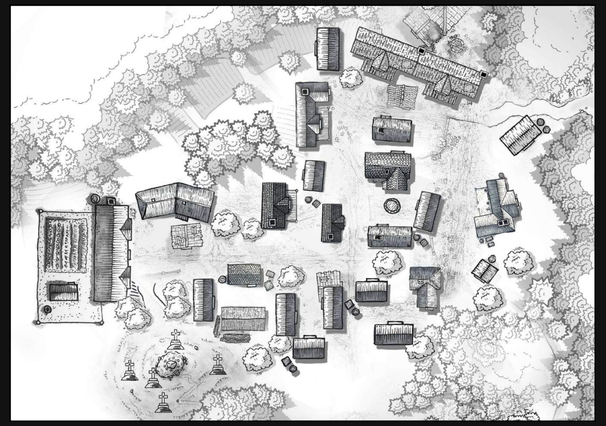
Dynamic Effects

Paper money blowing in the wind
Tattered cloth fluttering
Crows taking off suddenly
Thin smoke rising (early phase)
Lamps extinguishing (later phase)














Space & Environmental Storytelling

References



Syringes & Blood Bags

“Plasma 200ml”
“For Blood Station Use Only“
“Reuse to cut costs.”
“Danger! Source of infection.”

AIDS Patients’ Daily Items

“Painkillers”
“Antibiotics”
“Do not approach.”

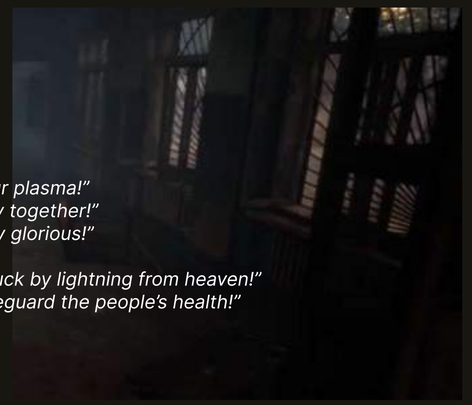
Propaganda Slogans & Banners

“Get rich by selling blood, a glorious contribution!”
If you want to achieve prosperity, hurry and sell your plasma!”
“Actively donate blood to build a prosperous society together!”
“One person donating blood makes the whole family glorious!”
“Prevent AIDS, cherish life!”
“Anyone who harms the common folk should be struck by lightning from heaven!”
“Crack down harshly on illegal blood collection, safeguard the people’s health!”







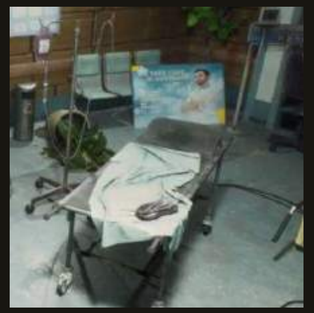






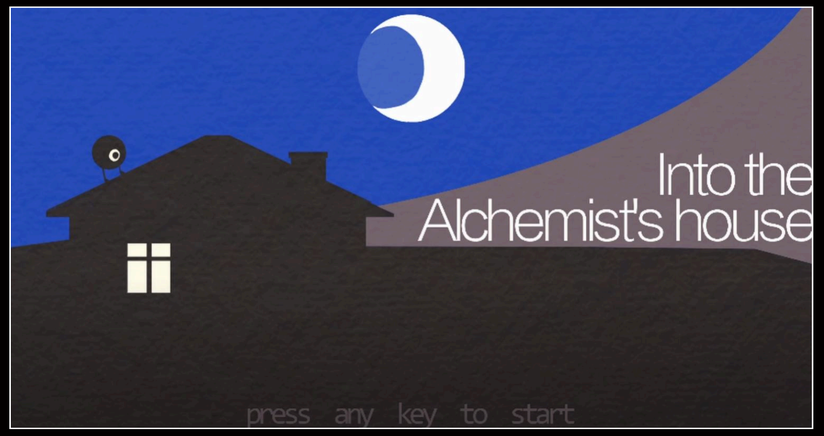












Duration: 7.17-7.29 (2024)
Genre: Puzzle Game
Gamejam Theme: Into the Alchemist's House
Platform: PC, Web
Team Size: 2

My Contribution: Game Designer & Level Designer
Video Link:
Demo Link:

Bio:
2d puzzle platformer, using drag and drop layers to change the lighting structure of the scene, and then change the structure of the scene to help the character realize the path.

The Concept:
Light& Dark,
Visable & Disappear
Collidable & non-Collidable

Platform Properties

Regular Platforms

Always visible and interactable, regardless of lighting conditions. Collides in both light and dark conditions.



Light-visible Platforms

Only visible and interactable under light; disappear and become non-collidable in darkness.



Dark-visible Platforms

Only in the dark will be displayed because of contrast and collision, and will disappear in the light.



Platform Movement

Platforms have three movement patterns: looping, one-way movement, and automatic return.



Layer Properties

Layer Dragging

Players adjust the interaction between the light sources and the luminous objects in the foreground by dragging the X-axis position of the foreground layer.

Luminous Objects

All luminous objects are charged through connections to the moon, linked by electrical wires. When the moon touches these wires, it triggers electricity.



foreground

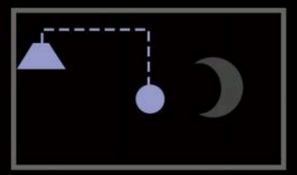


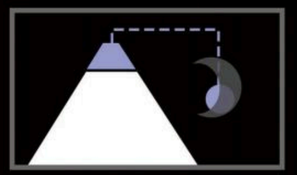
background



The game primarily consists of foreground and background layers. The foreground contains most platforms, while the background includes objects related to light sources.

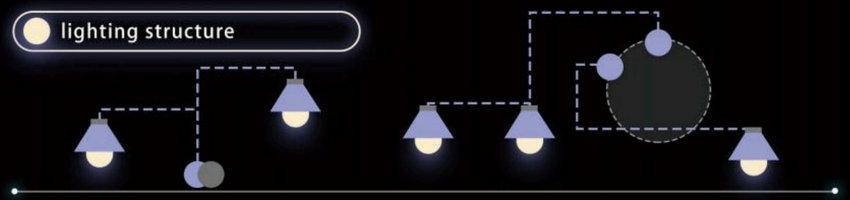
To maintain simplicity and coherence in gameplay, I avoid direct control of luminous objects like lamps through character interaction or cursor clicking. This design choice helps prevent a disconnect between first-person control and a god-like third-person perspective. Moreover, by limiting controls primarily to movement and dragging, we keep the interface clean and focused, avoiding unnecessary complexity that could clutter the gameplay experience.





Layer Position

Different layer positions affect different luminous bodies. Due to electrical connections, luminous bodies may form serial or parallel circuits, requiring precise adjustments by the player to construct the desired lighting structure.



Timing of Light Activation

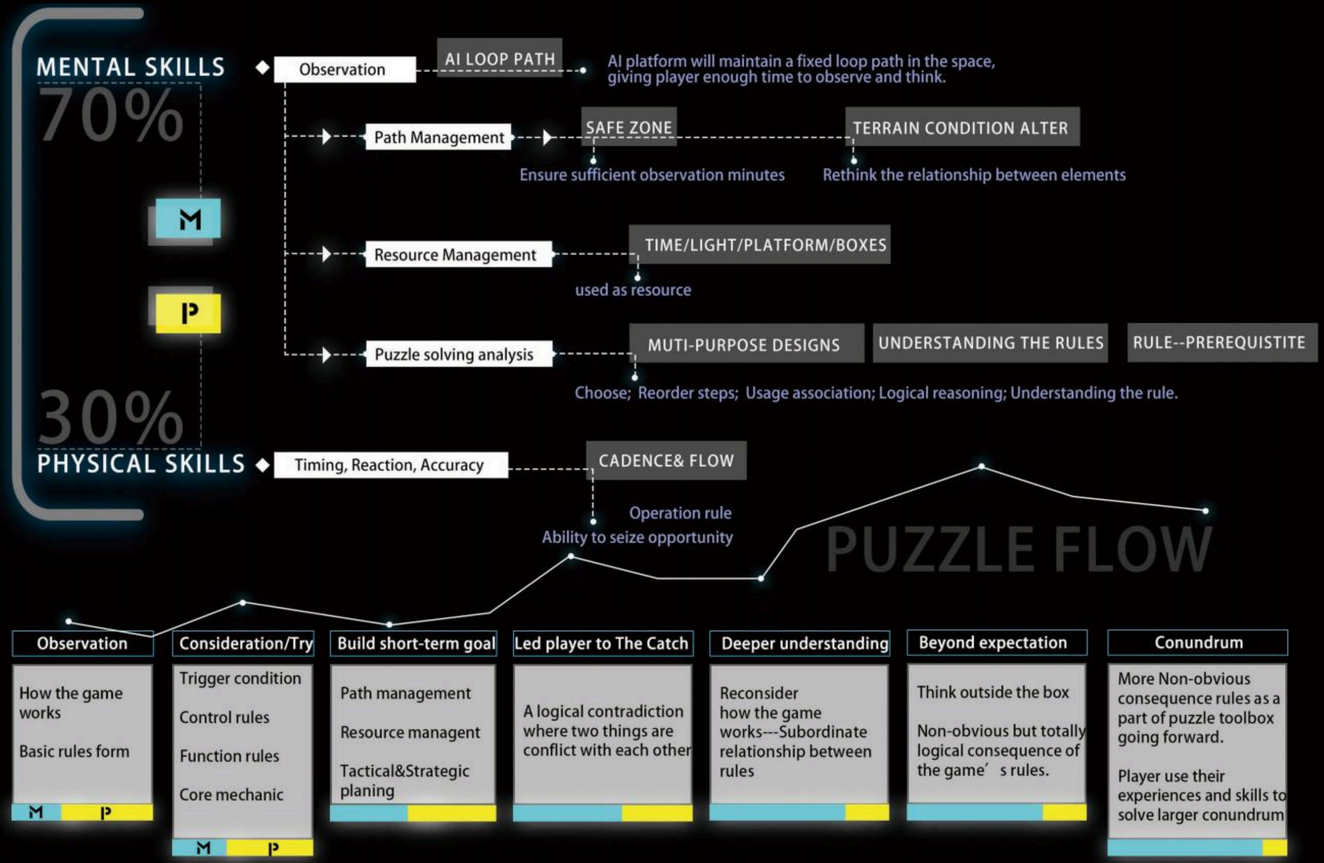
Players need to determine the timing of light activation based on the construction of paths, involving physical actions such as triggering lights while jumping in the air or before a platform disappears.



Sequence of Light Activation

Different activation sequences will layer various states of pathways, requiring players to use lighting to build platforms, avoid traps, and construct paths.







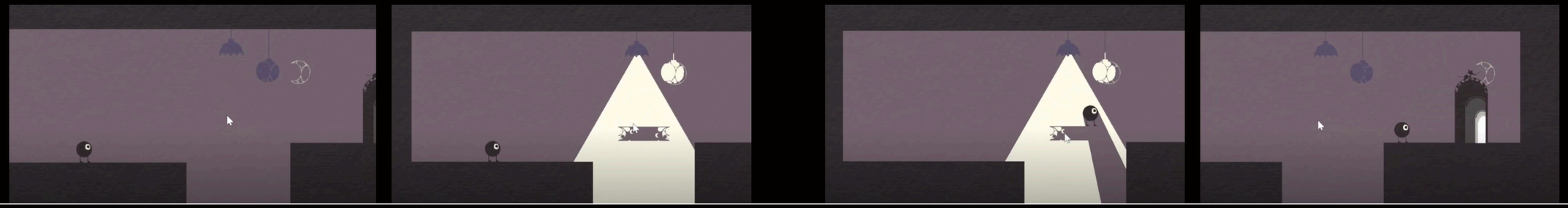
Design Concept: Explore the contrast between light and shadow, and how this contrast affects an individual's perception and reality.

Visual Art Style: Utilize strong black and white contrasts to create a somber yet dreamlike atmosphere, emphasizing the dramatic interplay between light and shadow.

Theme:This game's level design is centered around a theme of continuous exploration: the protagonist desires to escape a closed tower to admire the starry sky and moonlight. In the game, the moon is more than just a visual element; it serves a crucial mechanical function, symbolizing "electricity" and "light"



LEVEL 01-INTRO-LIGHT& SHADOW



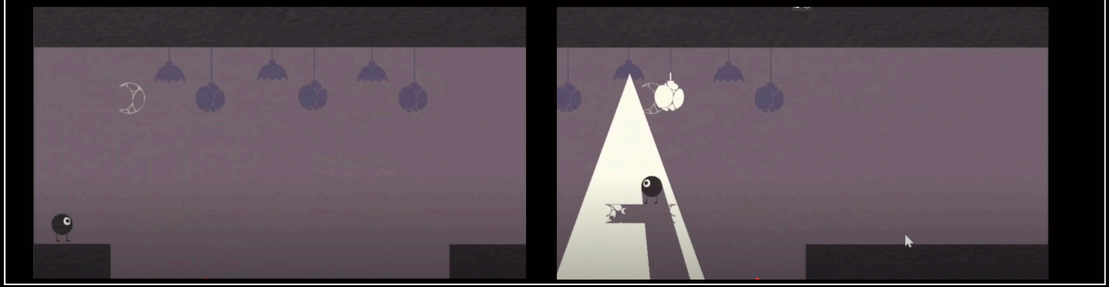
LEVEL 02-TRAIN



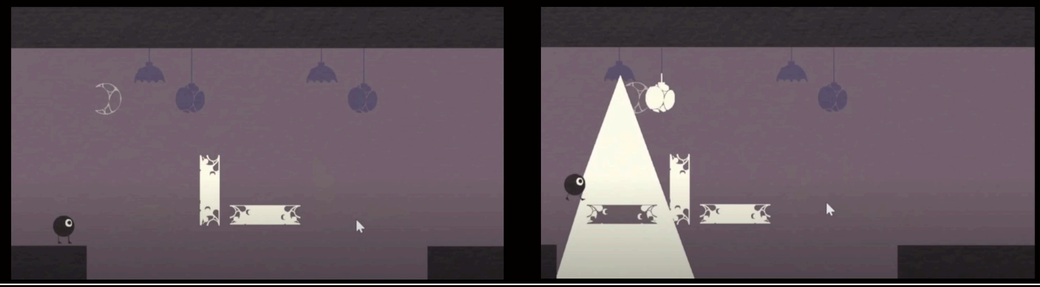
LEVEL 03-TWIST



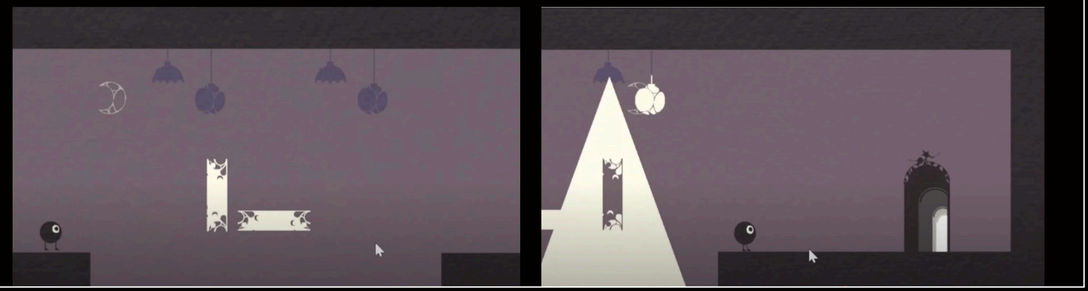
LEVEL 04-AHA MOMENT



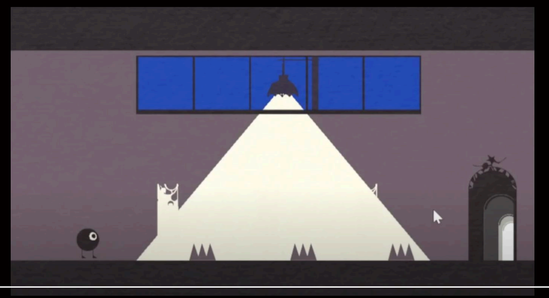
LEVEL 05-NEW ELEMENT INTRO



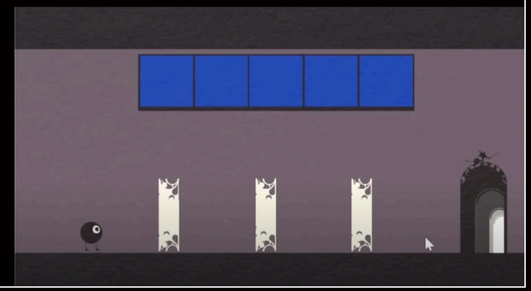
LEVEL 06-TWIST



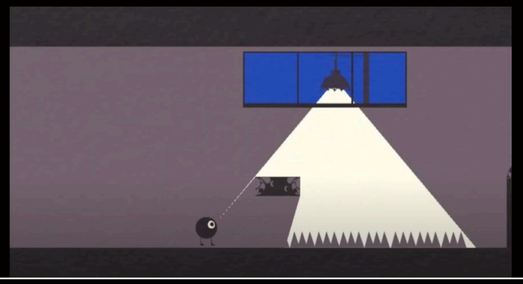
LEVEL 07-NEW MECHANIC

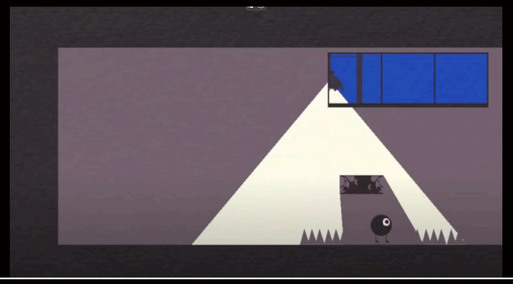


LEVEL 08-TWIST



LEVEL 09-NEW GAMEPLAY





LEVEL 10- TRAIN

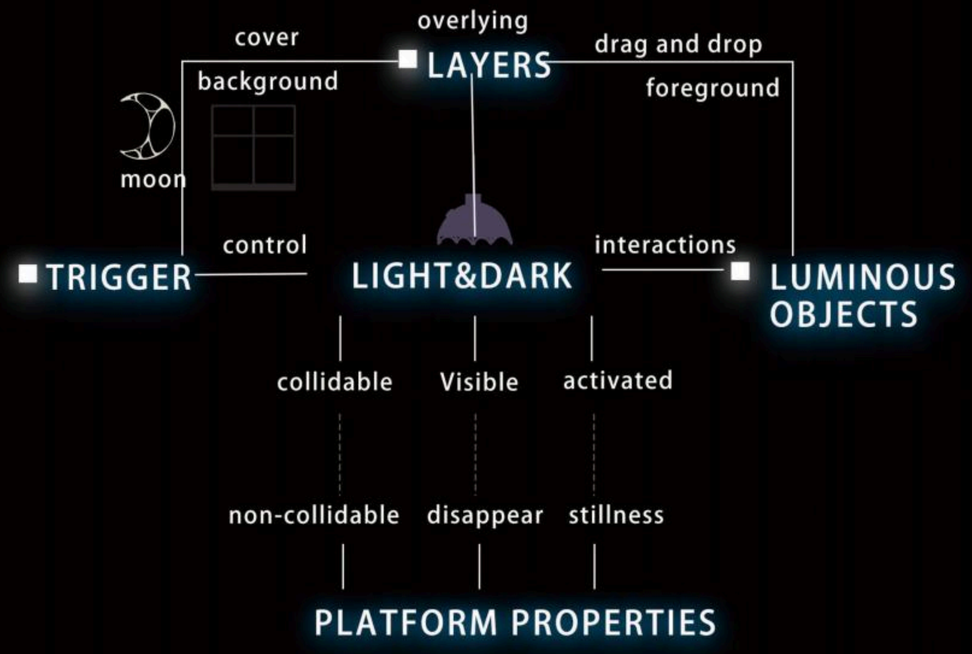


LEVEL-11 NEW GAMEPLAY



Dynamics & multi-purposes design of ingredients

The effects of ingredient interactions can be either positive or negative.



New elements may introduced into later levels

Boxes



◆ Regular boxes



◆ Light-visible boxes

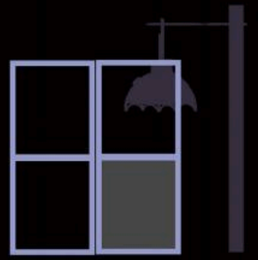


◆ Dark-visible boxes



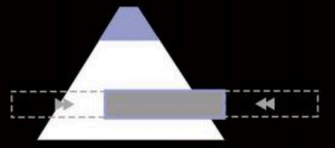
◆ Lighting boxes

Windows

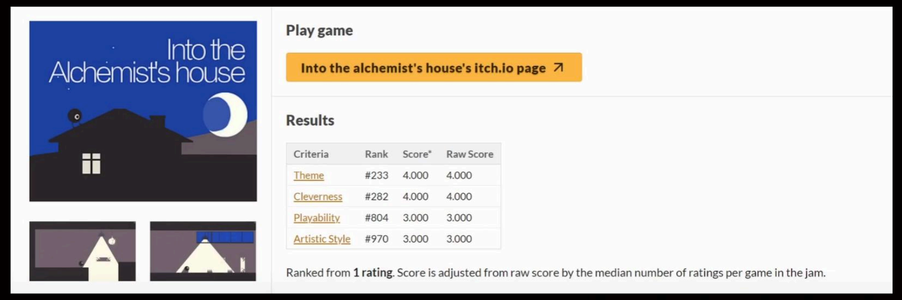


◆ Different range of occlusion
◆ Open and close state
◆ Obejcts behind the window

Platform



◆ Activated(move) under the light



CODE

This is achieved by calculating intersections of rays with line segments that represent obstacles. The rays are cast toward each endpoint of these segments, and a few additional rays are cast just beside each endpoint to ensure coverage around corners. The intersection points are then sorted by angle, allowing the creation of a visibility polygon which delineates the areas visible from the source point. This technique helps simulate realistic lighting and shadow effects in 2D environments.

Light source:

how to create 2D visibility and shadow effects in games:

The concept involves casting rays from a source (like a character in a game) to determine visible areas. Any line boundary can be written in parametric form as: Point + Direction * T,This gives us 4 equations describing the x & y components of a ray & line segment:

Ray X = r_px+r_dx*T1
Ray Y = r_py+r_dy*T1
Segment X = s_px+s_dx*T2
Segment Y = s_py+s_dy*T2

// Isolate T1 for both equations, getting rid of T1
T1 = (s_px+s_dx*T2-r_px)/r_dx =
(s_py+s_dy*T2-r_py)/r_dy

// Multiply both sides by r_dx * r_dy
s_px*r_dy + s_dx*T2*r_dy - r_px*r_dy = s_py*r_dx +
s_dy*T2*r_dx - r_py*r_dx

// Solve for T2!
T2 = (r_dx*(s_py-r_py) +
r_dy*(r_px-s_px))/(s_dx*r_dy - s_dy*r_dx)

// Plug the value of T2 to get T1
T1 = (s_px+s_dx*T2-r_px)/r_dx

```
void CastRays(bool isFullCircle)
{
    List<Vector3> points = new List<Vector3>();

    float angleStep = isFullCircle ? 360f / rays : (endAngle - startAngle) / (rays - 1);
    for (int i = 0; i < rays; i++)
    {
        float angle = isFullCircle ? i * 360f / rays : startAngle + i * angleStep;
        float radian = -angle * Mathf.Deg2Rad;
        Vector3 dir = new Vector3( x: Mathf.Cos(radian), y: Mathf.Sin(radian), z: 0);

        RaycastHit2D hit = Physics2D.Raycast( (Vector2) origin: transform.position, (Vector2) direction: dir, radius, (int) layerMask);
        if (hit.collider != null)
        {
            points.Add( (Vector3) hit.point);
            HandlePlatformHits( origin: transform.position, (Vector3) hit.point, dir);
        }
        else
        {
            Vector3 endPoint = transform.position + dir * radius;
            points.Add(endPoint);
            HandlePlatformHits( origin: transform.position, endPoint, dir);
        }
    }

    points.Sort( comparison: (a Vector3 , b Vector3 ) => Mathf.Atan2( y: a.y - transform.position.y, x: a.x - transform.position.x)
        .CompareTo(Mathf.Atan2( y: b.y - transform.position.y, x: b.x - transform.position.x)));

    int verticesCount = isFullCircle ? points.Count + 2 : points.Count + 1;
    vertices = new Vector3[verticesCount];
    triangles = new int[(points.Count) * 3];

    vertices[0] = Vector3.zero;
    for (int i = 0; i < points.Count; i++)
    {
        vertices[i + 1] = transform.InverseTransformPoint(points[i]);
    }

    for (int i = 0; i < points.Count - 1; i++)
    {
        triangles[i * 3] = 0;
        triangles[i * 3 + 1] = i + 1;
        triangles[i * 3 + 2] = i + 2;
    }
```

code for new collision for illuminated platforms

```
void UpdatePlatformCollisions()
{
    // Enable collision for illuminated platforms
    foreach (var platform GameObject in darkPlatforms)
    {
        Collider2D collider = platform.GetComponent<Collider2D>();
        if (collider != null)
        {
            collider.isTrigger = false;
            //Debug.Log("Enabling collision for: " + platform.name);
        }

        LightTriggerEvent lightTriggerEvent = platform.GetComponentInParent<LightTriggerEvent>();
        if (lightTriggerEvent != null)
        {
            lightTriggerEvent.onLightOn.Invoke();
        }
    }

    // Disable collision for platforms that are no longer illuminated
    foreach (var platform GameObject in previousDarkplatforms)
    {
        if (!darkPlatforms.Contains(platform))
        {
            Collider2D collider = platform.GetComponent<Collider2D>();
            if (collider != null)
            {
                collider.isTrigger = true;
                //Debug.Log("Disabling collision for: " + platform.name);
            }

            LightTriggerEvent lightTriggerEvent = platform.GetComponentInParent<LightTriggerEvent>();
            if (lightTriggerEvent != null)
            {
                lightTriggerEvent.onLightOff.Invoke();
            }
        }
    }
```

new boundary for scene

```
Event function  Dabu 10
void Start()
{
    mainCamera = Camera.main;

    // 创建并配置左边界
    leftBound = gameObject.AddComponent<BoxCollider2D>();
    leftBound.size = new Vector2( x: boundaryThickness, y: mainCamera.orthographicSize * 2);
    leftBound.offset = new Vector2( x: -mainCamera.orthographicSize * mainCamera.aspect - boundaryThickness / 2, y: 0);

    // 创建并配置右边界
    rightBound = gameObject.AddComponent<BoxCollider2D>();
    rightBound.size = new Vector2( x: boundaryThickness, y: mainCamera.orthographicSize * 2);
    rightBound.offset = new Vector2( x: mainCamera.orthographicSize * mainCamera.aspect + boundaryThickness / 2, y: 0);

    // 创建并配置上边界
    topBound = gameObject.AddComponent<BoxCollider2D>();
    topBound.size = new Vector2( x: mainCamera.orthographicSize * 2 * mainCamera.aspect, y: boundaryThickness);
    topBound.offset = new Vector2( x: 0, y: mainCamera.orthographicSize + boundaryThickness / 2);
}
```